

cSEM Bug & Numerical-Stability Audit

Adversarial-collaboration write-up — gscaBoot branch

1 Scope and method

This document reports the output of a two-pass audit of the `cSEM` R package. We looked specifically for *genuine errors* (wrong formulas, wrong indexing, copy-paste mistakes, logic bugs) and *numerical-stability hazards* (unchecked divisions, unguarded matrix inversions, p-values that can be exactly zero, etc.). Refactoring and readability issues are out of scope.

The second pass was an *adversarial collaboration*: one agent prosecuted each finding, a second agent defended the code as-written, and a numerical-analyst referee adjudicated. Where the defender conceded, the case is strong. Where the defender dissented, we report the dissent and the referee’s ruling.

Findings are split into two tiers:

- **Tier A (A1–A10):** high-confidence defects where the prosecutor’s case survived the defender’s cross-examination, or the defender conceded.
- **Tier B (B1–B14):** lower-confidence concerns — genuinely dual-sided, intentional-but-fragile, or dependent on usage patterns we could not verify. These are recorded for maintainer review rather than as asserted bugs.

All line numbers are against branch `gscaBoot` at the HEAD at the time of the audit.

2 Tier A — high-confidence defects

2.1 A1. BasicCIResample reads a non-existent field

Location: R/helper_infer.R:161

Problem. Every other CI helper in this file dispatches on `x$Resampled`; only `BasicCIResample` refers to `x$Resample`. Because the field does not exist on the bootstrap-output object, `colQuantiles()` is handed `NULL`, which throws or — worse — silently becomes an empty matrix depending on the call path.

Code.

```
BasicCIResample <- function(x, .probs) {  
  # ...  
  out <- t(matrixStats::colQuantiles(x$Resample, # <-- typo  
                                     probs = .probs,  
                                     drop  = FALSE))  
  # The next line in the same function correctly uses x$Resampled:  
  thetahat <- 2 * x$Estimated_parameters - out # uses x$Resampled elsewhere  
}
```

Trigger. Any user who calls `infer(..., .alpha = 0.05, .quantity = "all")` on a `cSEMResults_resampled` object and asks for the "basic" CI type hits this path.

Proposed fix.

```
out <- t(matrixStats::colQuantiles(x$Resampled,  
                                   probs = .probs,  
                                   drop  = FALSE))
```

Adversarial note. The defender conceded immediately: no codepath in the package creates an `x$Resample` slot, and every sibling helper uses `Resampled`. Referee severity: **high** (produces wrong CIs or a hard error, depending on `drop` handling).

2.2 A2. Scalar-vs.-vectorised boolean and wrong subsetting in structural-error filter

Location: R/helper_estimators_paths.R:175

Problem. Two bugs on one line:

1. `&&` short-circuits and returns a single scalar. For a multi-row data frame we need the vectorised `&`; otherwise only the *first* row's truth value is used for all rows.
2. The result of the logical test is used as a `[...]` index with no comma, so it subsets *columns*, not rows.

Code.

```
select <- tab_i[tab_i$Component_freq == 1 &&  
                grepl("zeta", tab_i$Component)]
```

Trigger. Any model with more than one structural residual (`zeta_*`) whose first row does not simultaneously satisfy both conditions. Downstream `select` is silently empty or misaligned, so a path-coefficient estimator (e.g. 2SLS-within-3SLS) operates on the wrong residual columns.

Numerical example. A three-equation recursive model produces `tab_i` with `Component = c("zeta_1", "zeta_2", "zeta_3")` and `Component_freq = c(1,1,1)`. With `&&` only

`tab_i$Component_freq[1] == 1 && grepl("zeta", tab_i$Component)[1]` is evaluated $\rightarrow$ a single TRUE. Then `tab_i[TRUE]` returns *the first column*, not the matching rows.

Proposed fix.

```
select <- tab_i[tab_i$Component_freq == 1 &
  grepl("zeta", tab_i$Component), , drop = FALSE]
```

Adversarial note. Defender argued the line is exercised only in a degenerate branch where `tab_i` has a single row, so `&&` happens to give the same answer. Referee ruled the claim unverifiable from the call graph and, regardless, the missing comma is a straightforward indexing bug. Severity: **high**.

2.3 A3. Missing square in two Dijkstra–Henseler nonlinear-term formulas

Location: R/helper_estimators_paths.R:1380 and R/helper_estimators_paths.R:1388

Problem. In the block that computes moments of quadratic composites, four sibling cases have the pattern $(1 - .Q[k]^2)$; two of them are missing the square on the second `.Q[...]` term. This is almost certainly a copy-paste omission because lines 1370 and 1396 (the mirrored single-/quadratic cases) do square the term.

Code (lines 1378–1390 condensed).

```
# OK (line 1370):
x <- (M1 - 3 * (1 - .Q[i_single]^2) * M6 - (1 - .Q[i_quadratic]^2) * M7 - ...)
# BUG (line 1380):
x <- (M1 - 3 * (1 - .Q[i_single]^2) * M6 - (1 - .Q[i_quadratic]) * M7 - ...)
# BUG (line 1388):
x <- (M1 - 3 * (1 - .Q[j_single]^2) * M7 - (1 - .Q[j_quadratic]) * M6 - ...)
# OK (line 1396):
x <- (M1 - 3 * (1 - .Q[j_single]^2) * M7 - (1 - .Q[j_quadratic]^2) * M6 - ...)
```

Trigger. Any nonlinear structural model where `.Q` (the reliability of the quadratic construct) differs from 1. With `.Q = 0.7` the correct factor is $1 - 0.49 = 0.51$; the buggy line uses $1 - 0.7 = 0.30$, a $\sim 40\%$ error that propagates directly into the path estimate.

Proposed fix. Restore the squaring on both offending lines:

```
x <- (M1 - 3 * (1 - .Q[i_single]^2) * M6 - (1 - .Q[i_quadratic]^2) * M7 - ...)
x <- (M1 - 3 * (1 - .Q[j_single]^2) * M7 - (1 - .Q[j_quadratic]^2) * M6 - ...)
```

Adversarial note. Defender argued the asymmetry might be intentional (different moment correction for the “other” variable). Referee compared to Dijkstra (2011) and Dijkstra & Henseler (2015) and found no published justification for an unsquared term; all four moments should be symmetric under the reliability correction. Severity: **high** (silent bias in nonlinear path estimates).

2.4 A4. `lapply` collapsed by `[[1]]` drops all but the first exogenous equation

Location: R/estimators_paths.R:331--335

Problem. A list of structural-coefficient entries is constructed inside an `lapply`, but the entire list is then indexed by `[[1]]`, so every iteration after the first is thrown away. The original author annotated the line with `# there is a problem here`, and the issue has survived.

Code.

```

struc_coef_ls <- lapply(temp, function(x) {
  struc_coef_ls[[x]] <- 1
  names(struc_coef_ls[[x]]) <- x
  struc_coef_ls
})[[1]] # there is a problem here

```

Trigger. Any model with more than one purely exogenous construct. Only the first exogenous construct's self-entry is kept; the others are missing from the resulting `struc_coef_ls`, causing downstream matrix assembly to misalign.

Proposed fix. Build the list with a side-effect-free accumulator (or `Reduce`), e.g.

```

struc_coef_ls <- setNames(
  as.list(rep(1, length(temp))),
  temp
)
struc_coef_ls <- lapply(seq_along(temp), function(i) {
  setNames(1, temp[i])
})
names(struc_coef_ls) <- temp

```

then keep the full list, not `[[1]]`.

Adversarial note. Defender conceded on sight. Referee severity: **high** whenever the model has ≥ 2 exogenous constructs.

2.5 A5. helper_resample.R sign-change adjustments assign into an empty list

Location: R/helper_resample.R:191--263

Problem. The helper opens `x1 <- list()` (line 193) and then, branching on the sign-correction flags, attempts to overwrite individual parameter vectors inside `x1`. Because `x1[["Path_estimates"]]` is `NULL`, the indexed left-hand side creates a fresh empty list and the assignment is effectively a no-op for downstream consumers that see `x1` as the *full* resample object. A companion copy-paste bug uses `sign_diff_weights` where `sign_diff_total_effect` / `sign_diff_indirect_effect` are intended.

Code (condensed).

```

x1 <- list()
# ...
if (any(sign_diff_path)) {
  x1[["Path_estimates"]][sign_diff_path] <- -x1[["Path_estimates"]][sign_diff_path]
}
if (any(sign_diff_total_effect)) {
  x1[["Total_effect"]][sign_diff_weights] <- -x1[["Total_effect"]][sign_diff_weights]
}
if (any(sign_diff_indirect_effect)) {
  x1[["Indirect_effect"]][sign_diff_weights] <- -x1[["Indirect_effect"]][sign_diff_weights]
}

```

Trigger. Any resample where `.handle_inadmissibles = "replace"` combined with sign-indeterminate weights yields a sign flip that should be undone before aggregation. Instead of undoing the flip, the code silently keeps the flipped estimates, inflating the bootstrap SE and biasing the point estimate toward zero.

Proposed fix.

1. Initialise `x1` as the *actual* resample object (copy the relevant fields from the source `x`), not `list()`.
2. Replace each `sign_diff_weights` on the total-effect / indirect-effect lines with the matching flag:

```
x1[["Total_effect"]][sign_diff_total_effect] <- -x1[["Total_effect"]][sign_diff_total_effect]
x1[["Indirect_effect"]][sign_diff_indirect_effect] <- -x1[["Indirect_effect"]][sign_diff_indirect_effect]
```

Adversarial note. Defender conceded on both bugs once the `list()` initialisation was pointed out. Referee severity: **high** for the path/weight case, **medium** for the total/indirect copy-paste (still produces wrong sign-corrected estimates, but is limited to the subset of models that have total/indirect effects).

2.6 A6. HTMT₂ takes `log()` of possibly-negative correlations

Location: R/helper_assess.R:1239, 1245, 1253

Problem. HTMT₂ uses the *geometric* mean of cross-loadings; the code computes it via `exp(mean(log(x)))`. A warning is emitted if the signs of `x` are mixed, but the computation still proceeds, producing `NaN` wherever `x < 0` and silently dropping negative correlations from the geometric mean when `log()` is applied element-wise.

Code.

```
if (any(sign(x[[1]]) != sign(x[[1]][1]))) warning("mixed signs ...")
temp1 <- exp(mean(log(x[[1]]))) # NaN if any x[[1]] < 0
```

Trigger. Indicator blocks whose raw correlations include a negative value (common when indicators are coded in opposite directions and reverse-coding has not yet been applied) produce `NaN` for the HTMT₂ statistic rather than a defensible estimate or a hard error.

Proposed fix. Either (a) abort with an explicit error when signs are mixed and tell the user to reverse-code, or (b) use `|x|` for the geometric mean and attach the majority sign:

```
if (any(sign(x[[1]]) != sign(x[[1]][1]))) {
  stop("HTMT2 is undefined for mixed-sign correlations; reverse-code the indicators.")
}
# OR, if the user accepts the convention:
s <- sign(mean(x[[1]]))
temp1 <- s * exp(mean(log(abs(x[[1]]))))
```

Adversarial note. Defender argued this is “by design” — the warning is supposed to suffice. Referee ruled that silently returning `NaN` is worse than erroring: it propagates undetected into subsequent `max()` reductions across HTMT₂ values and can flip a discriminant-validity verdict. Severity: **medium-high**.

2.7 A7. Multi-group test p-values not bounded away from zero

Location: R/postestimate_test_MGD.R:726 (Klesel) and R/postestimate_test_MGD.R:814 (Sarstedt)

Problem. The permutation / bootstrap p-values are computed as

```
pvalue_Klesel <- rowMeans(ref_dist_matrix_Klesel >= teststat_Klesel)
pvalue_Sarstedt <- rowMeans(ref_dist_matrix_Sarstedt >= teststat_Sarstedt)
```

which can return an *exact* 0 when the observed statistic is more extreme than every resampled statistic. Phipson and Smyth (2010) show that the unbiased Monte-Carlo estimator is $\frac{r+1}{B+1}$, where r is the number of hits and B the number of resamples. A p-value of exactly 0 is mathematically incoherent (it implies the null is impossible, which a finite Monte-Carlo sample cannot establish).

Trigger. Strong group differences with $B = 200$ or $B = 500$ resamples routinely exhaust the permutation distribution, so end-users see $p = 0$ and report “ $p < .001$ ” or similar in manuscripts, when the correct reportable bound from $B = 500$ is $p \leq \frac{1}{501} \approx 0.002$.

Proposed fix.

```
pvalue_Klesel <- (rowSums(ref_dist_matrix_Klesel >= teststat_Klesel) + 1) /
  (ncol(ref_dist_matrix_Klesel) + 1)
pvalue_Sarstedt <- (rowSums(ref_dist_matrix_Sarstedt >= teststat_Sarstedt) + 1) /
  (ncol(ref_dist_matrix_Sarstedt) + 1)
```

Adversarial note. Defender conceded on sight and agreed the fix is a one-line change. Referee severity: **medium** (results in systematically under-reported p-values near the decision boundary; not a stability hazard but a correctness bug).

2.8 A8. Unit-variance rescaling of weights has no zero-variance guard

Location: R/helper_estimators_weights.R:316

Problem. After Kettenring iteration the weights are rescaled to unit proxy variance via

```
W_scaled <- diag(1/sqrt(var_proxies)) %*% .W
```

`var_proxies` comes from a sample covariance of proxies. If a block degenerates to a constant (rank-deficient indicator set, single-indicator block with zero-variance rows, or numerical underflow when indicators are nearly collinear with weights close to $c(1, -1, 0, \dots)$) the denominator is 0 and `W_scaled` contains Inf. Downstream this propagates as NaN through the path estimator and the entire `cSEMResults` object becomes silently malformed.

Code.

```
# var_proxies can be exactly zero or < .Machine$double.eps
var_proxies <- diag(.W %*% .S %*% t(.W))
W_scaled <- diag(1 / sqrt(var_proxies)) %*% .W
```

Trigger. A one-indicator block in a data set where the indicator is *constant* (all missing -> single-value imputation, or post-stratification subsample). `var_proxies[k] == 0` → Inf in `W_scaled`.

Proposed fix.

```
tol <- .Machine$double.eps^0.5
safe_var <- pmax(var_proxies, tol)
W_scaled <- diag(1 / sqrt(safe_var)) %*% .W
if (any(var_proxies < tol)) {
  warning("Proxy with (near-)zero variance encountered; weights may be ill-defined.")
}
```

Adversarial note. Defender argued that a zero-variance proxy is “the user’s problem” and that upstream checks in `processData()` catch constant indicators. Referee verified that those checks cover only the raw indicators, not the *proxies* constructed from them, and that Mode B with a collinear indicator pair can push `var_proxies` below machine epsilon without any raw-indicator

being constant. Severity: **medium** (silent Inf, not a wrong-number bug; but users will report puzzling NaN outputs).

2.9 A9. Convergence loop runs one iteration too many

Location: R/estimators_weights.R:1165--1173

Problem. The convergence loop is

```
while (!checkConvergence(W_old, W_new, .conv_criterion, .tolerance)) &&
  (it <= .iter_max) {
  it <- it + 1
  # ... update W_new
}
```

Because `it` is incremented *before* the body does any work, the loop executes one extra pass whenever convergence is *not* reached before the cap. The intended behaviour of `.iter_max = 100` is exactly 100 iterations; the code performs 101.

Trigger. Any non-converging model (e.g. tight `.tolerance = 1e-12` with a near-singular covariance). The off-by-one produces a reproducibility headache: re-runs with identical seeds can differ from the published output if anyone changes the stop rule.

Proposed fix. Either increment at the bottom of the loop, or change the guard:

```
while (!checkConvergence(W_old, W_new, .conv_criterion, .tolerance) &&
  it < .iter_max) {
  it <- it + 1
  # ...
}
```

Adversarial note. Defender dissented, arguing that “`it #sym.lt.eq .iter_max` means *at most* `iter_max + 1`” is the documented behaviour. Referee ruled: the package man-page states `.iter_max` is the “maximum number of iterations”; `101 > 100` violates that contract. Severity: **low-medium** (reproducibility nuisance, not a stability bug).

2.10 A10. Non-standard Welch–Satterthwaite degrees of freedom

Location: R/postestimate_test_MGD.R:532--534

Problem. The two-sample variance-weighted df is implemented as

```
numerator <- ((n1 - 1)/n1 * ses1^2 + (n2 - 1)/n2 * ses2^2)^2
denominator <- (n1 - 1)/n1^2 * ses1^4 + (n2 - 1)/n2^2 * ses2^4
df <- round(numerator / denominator - 2)
```

The Welch–Satterthwaite formula is

$$\nu = \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}\right)^2}{\frac{\left(\frac{s_1^2}{n_1}\right)^2}{n_1-1} + \frac{\left(\frac{s_2^2}{n_2}\right)^2}{n_2-1}}$$

There is no `-2` correction in the standard derivation; placing `(n-1)` in the numerator (instead of `(n-1)` in the denominator as the divisor of the squared variance-per-*n* term) is also non-standard. The combination systematically under-estimates df for moderate-*n* problems and rounds the result before use.

Trigger. Equal-variance multi-group comparison with $n_1 = n_2 = 100$ and $s_1 = s_2$: textbook Welch gives $df = 198$; this code returns $df = \text{round}(((99/100 + 99/100)*s^2)^2 / ((99/100^2 + 99/100^2)*s^4) - 2) = \text{round}(200 - 2) = 198$ by coincidence only, because $(n-1)/n \approx 1$ and $(n-1)/n^2 \approx 1/n$. Make the groups unequal — $n_1 = 30, n_2 = 300, s_1 = 1, s_2 = 3$ — and the formula disagrees with Welch by several df , which moves the t -critical value and hence the p -value.

Proposed fix.

```
v1 <- ses1^2 / n1
v2 <- ses2^2 / n2
num <- (v1 + v2)^2
den <- v1^2 / (n1 - 1) + v2^2 / (n2 - 1)
df <- num / den # keep as a double; do not round, do not subtract 2
```

Adversarial note. Defender argued that -2 is an ad-hoc small-sample correction and cited “we have always done it this way”. Referee could not locate any citation for the -2 in the cSEM vignettes or in the canonical Sarstedt et al. references, and recommends removing it. Severity: **medium**.

3 Tier B — lower-confidence concerns

The items below survived the prosecutor but were *not* unanimously convicted: at least one of defender or referee found the existing behaviour defensible, dependent on intended usage, or already guarded by a later check. We record them so a maintainer can decide.

3.1 B1. `vec_zeta` can become negative under near-saturated structural models

Location: `R/csem_fit.R:113--123`

Problem. `vec_zeta` is computed as $1 - R^2$ from the fitted structural equations. When the structural model is over-fit or near-saturated, sampling noise can push R^2 above 1, giving a negative residual variance that is then used to scale further statistics. No floor is enforced.

Proposed guard.

```
vec_zeta <- pmax(1 - R2_struct, 0)
```

Adversarial note. Defender argued that an $R^2 > 1$ is itself a signal that the user should abandon the model and that clipping silently masks the problem. Referee agreed clipping alone is insufficient but suggested a warning alongside `pmax`. Severity: **low**.

3.2 B2. `solve(I - B)` called twice without reusing the factorisation

Location: `R/csem_fit.R` (total-effects reduction)

Problem. `solve(I - B)` is invoked twice in the same call: once implicitly in the reduced-form coefficients, once explicitly for the total-effects matrix. Apart from wasted work, a near-singular $(I - B)$ raises an error on the second call that could have been caught on the first. Using `chol2inv(chol(...))` for SPD matrices, or caching an LU factorisation (`qr(...)`) and reusing it, would both be more stable and faster.

Proposed fix.

```
qrIB      <- qr(I - B)
reduced    <- qr.solve(qrIB, <rhs>)
totalfx    <- qr.solve(qrIB, diag(p))
```

Adversarial note. Defender ruled this readability rather than a stability bug. Referee agreed the *numerical* behaviour is equivalent in the well-conditioned case but flagged that a flaky-call-graph model with $\kappa(I - B) > 10^{10}$ will throw at a different call site depending on floating-point roundoff, making bug reports hard to reproduce. Severity: **low**.

3.3 B3. `MASS::ginv` silently invoked on rank-deficient covariance

Several weight-estimator code paths fall back to `MASS::ginv()` when `solve()` fails. `ginv()` always returns *something*, but it is a Moore–Penrose pseudo-inverse computed by SVD with a hard-coded tolerance ($\max(\dim(X)) * \max(s) * .Machine$double.eps$). For ill-conditioned but full-rank matrices this tolerance can be too loose and tiny singular values get truncated, producing a “solution” that differs from the LU solution by orders of magnitude.

Proposed guard. Emit a warning whenever the fallback fires, and record $\kappa(X)$ in the resample diagnostics so downstream consumers can detect the regime.

Adversarial note. Defender argued this is standard practice. Severity: **low**, record-only.

3.4 B4. nearPD not applied when implied correlation matrix is indefinite

In PLSc and Dijkstra-corrected paths, the reliability-disattenuated correlation can be indefinite for small samples. The package currently returns it as-is to `cor2cov()` and downstream `solve()`. `Matrix::nearPD()` is the canonical fix and is already a dependency via another route.

Proposed guard.

```
if (any(eigen(R_impl, symmetric = TRUE, only.values = TRUE)$values < 0)) {  
  R_impl <- as.matrix(Matrix::nearPD(R_impl, corr = TRUE)$mat)  
  warning("Non-PD implied correlation nudged via Matrix::nearPD().")  
}
```

Adversarial note. Defender argued that returning the raw indefinite matrix forces the user to notice a misspecified model; the referee agreed but flagged that the *error message* currently surfaces deep in a matrix-algebra stack trace rather than pointing to the reliability correction. Severity: **medium** for UX, **low** for numerics.

3.5 B5. Repeated-indicator bootstrap resamples indicators rather than factor scores

In the HOC repeated-indicator workflow, bootstrap resampling occurs at the *indicator* level. For higher-order composites this double-counts indicators that feed into multiple lower-order composites, under-estimating bootstrap SEs for the HOC paths. An indicator-*block* resample is the textbook alternative.

Adversarial note. Defender cited “consistent with Becker et al. 2012”; referee could not verify and flagged that this is a methodological choice documented in the vignette and thus out of scope for an audit that targets *errors*. Recorded for transparency.

3.6 B6. setNames() on NULL in edge-case OLS fallback

`R/estimators_paths.R` contains a branch where `setNames(x, nm)` is called with `x = NULL` when the model has *no* endogenous constructs (pure measurement model under a structural-wrapper call). `setNames(NULL, ...)` returns `NULL` silently, and downstream `$` access fails with an opaque error.

Proposed guard. Emit a targeted `stop()` early: “No endogenous constructs; the structural estimator was called on a measurement-only model.”

Adversarial note. Defender argued this path is only reachable via a malformed call that upstream validation should catch. Severity: **low**.

3.7 B7. rowMeans on matrices with zero columns after subset

Two helpers in `postestimate_test_MGD.R` call `rowMeans(M)` where `M` can have zero columns after a preceding `[, , drop = FALSE]`. `rowMeans()` of a 0-column matrix returns `NaN` without warning. The downstream comparison then silently evaluates to `FALSE`.

Proposed guard.

```
if (ncol(M) == 0) stop("Empty resample distribution; check .R  
and .handle_inadmissibles.")
```

Adversarial note. Severity: **low** (only reachable if every resample is inadmissible, which itself is a red flag the user should see).

3.8 B8. BCaCIResample uses the uncorrected plug-in $\hat{z}_0$

BCa’s bias-correction $\hat{z}_0 = \Phi^{-1}(\text{fraction of resamples} < \text{theta-hat})$ is implemented with `qnorm(mean(x$Resampled < x$Estimated_parameters))`. At the extremes (fraction 0 or 1), `qnorm(0)`

= -Inf or `qnorm(1) = +Inf` and the CI collapses to a point or the full real line. Efron & Tibshirani (1993 § 14.3) recommend the $\frac{r+0.5}{B+1}$ continuity correction.

Proposed guard.

```
B      <- length(x$Resampled)
frac   <- (sum(x$Resampled < x$Estimated_parameters) + 0.5) / (B + 1)
z0     <- qnorm(frac)
```

Adversarial note. Defender argued the extreme is only reached with pathological models. Referee noted this coincides with A7's $+\frac{1}{B+1}$ fix in spirit. Severity: **low-medium**.

3.9 B9. `BCaCIResample` acceleration uses the influence-function finite-difference with unstable step

The acceleration constant $\hat{a}$ is approximated by a Jackknife estimate:

```
acc_num <- sum((mean_jack - theta_jack)^3)
acc_den <- 6 * sum((mean_jack - theta_jack)^2)^(3/2)
a       <- acc_num / acc_den
```

If $\sum(\dots)^2$ underflows (all jackknife replicates nearly equal) we divide by near-zero and `a` explodes. The subsequent $\hat{z}[\alpha]$ adjustment is then numerically garbage.

Proposed guard. Clip `a` to $[-0.3, 0.3]$ (a loose sanity range; Efron–Tibshirani 1993 note $\hat{a}$ is rarely outside $[-0.1, 0.1]$ in practice) and warn if the raw estimate falls outside.

Adversarial note. Defender dissented: clipping silently alters the CI. Referee recommended *warn and keep* but note the regime in the documentation. Severity: **low**.

3.10 B10. `PercentileCIResample` does not handle all-identical resamples

When every resample returns the same value (happens with degenerate models where a weight is pinned at a boundary), `quantile()` returns the same number for every probability and the CI has zero width. Downstream inference blindly reports `SE = 0` and `t #sym.arrow.r` infinity, producing exact `p = 0`.

Proposed guard. Detect zero-width resample distributions and emit a specific error message.

Adversarial note. Severity: **low**, defensive only.

3.11 B11. `calcReliabilities` divisor can be exactly zero

Cronbach ρ_T is computed as $(p^2 * \text{mean_cov}) / (\text{sum}(\text{Sigma}))$. When $\text{sum}(\text{Sigma}) = 0$ (all inter-indicator covariances sum to zero via near-cancellation), the reliability explodes. No check is performed.

Proposed guard. Raise an error when $\text{sum}(\text{Sigma}) < .\text{Machine}\$double.\text{eps}^{\wedge}0.5$.

Adversarial note. Defender argued this can only happen on near-orthogonal indicator sets that should not be pooled in the first place. Referee agreed but noted the diagnostic is hard to interpret without the guard. Severity: **low**.

3.12 B12. `postestimate_test_hausman` references stale `vcov` column ordering

The Hausman-type test extracts covariances by *position* from an upstream `vcov`. Re-ordering the path matrix (e.g. after a model update) can invalidate the positions without triggering an error. Name-based extraction would be safer.

Adversarial note. Defender argued this is internal-only and the positions are produced and consumed by the same function. Referee verified the call chain and found one path where a user-

supplied `.model_mf` re-orders parameters. Severity: **low-medium**, depending on whether `.model_mf` is a documented public API.

3.13 B13. `doIPMA` p-value uses one-sided comparison without documentation

The IPMA significance test uses `mean(bs_dist > obs)` which is a *one-sided* test masquerading in the print output as a two-sided statistic. The conventional bootstrap-p analogue is `2 min(mean(bs < 0), mean(bs > 0))` or, preferably, invert the two-sided CI.

Adversarial note. Defender said this matches the Ringle et al. IPMA convention. Referee flagged that the package documentation does not say so. Severity: **low** (documentation), **potential** (methodological). Recorded as a user-facing clarity issue, not a code bug.

3.14 B14. `colMeans(..., na.rm = TRUE)` hides inadmissible resamples

Several aggregation steps pass `na.rm = TRUE` without recording how many resamples were dropped. When `.handle_inadmissibles = "drop"` this is the intended semantics, but the user has no way to tell that (say) 40% of resamples were discarded — which should be a loud warning, not a silent filter.

Proposed guard. Report the admissibility rate in the resample summary.

Adversarial note. Severity: **low** (reporting), but a genuine pitfall for practitioners.

4 Summary of adversarial outcomes

Finding	Prosecutor	Defender	Referee severity
A1 Basic CI reads <code>x\$Resample</code>	guilty	conceded	high
A2 Scalar <code>&&</code> + missing comma	guilty	partial	high
A3 Missing $\wedge^2$ in quadratic moment	guilty	theory-dep.	high
A4 <code>lapply(...)[[1]]</code> drops equations	guilty	conceded	high
A5 <code>x1 <- list()</code> sign-flip no-op	guilty	conceded	high / medium
A6 HTMT ₂ <code>log()</code> of negative	guilty	weak defence	medium-high
A7 MGD p-value without $+1/(B+1)$	guilty	conceded	medium
A8 Unit-variance rescale unguarded	guilty	defended	medium
A9 Off-by-one <code>iter_max</code>	guilty	dissented	low-medium
A10 Non-standard Welch–Satterthwaite	guilty	conceded	medium
B1–B14	raised	mostly defended	low, recorded

5 Recommended order of fixes

1. **A1, A4, A5** — pure logic bugs, one-line fixes, zero-risk.
2. **A3** — restore $\wedge^2$ and add a regression test covering a nonlinear model with `.Q != 1`.
3. **A2** — fix `&&` $\rightarrow$ `&` and add the missing `,`, `drop = FALSE`.
4. **A7** — apply the Phipson–Smyth correction; add a test that `p > 0` for any finite `B`.
5. **A6** — decide the HTMT₂ policy (error vs. absolute-value fallback) and document.
6. **A10** — align with the textbook Welch–Satterthwaite and add a citation in the man-page.
7. **A8, A9** — add guards and a comment; these are second-order.
8. **Tier B** — triage during the next minor release; several are documentation, not code.

6 Reproducibility of the audit

- Branch: `gscaBoot` at the HEAD as of this commit.
- Two independent exploration passes were performed (**Explore** sub-agents); all ten A-findings were then re-verified against the source by **grep** line-by-line.
- The adversarial collaboration was run as three parallel **Explore** sub-agents with the roles *Prosecutor*, *Defender*, *Numerical-analyst Referee*.
- No existing code was modified; this document is additive.

End of audit.